Semantic Skill Space (SSS): Latent Skill Injection for Small Language Models Under Context Constraints

Ethan Blair

March 2026
DOI: 10.5281/zenodo.18830835

Abstract

Skill files are widely used in agent systems, but they remain context-bound: they consume prompt tokens and are semantically dense in natural language form. This project tests a latent alternative, Semantic Skill Space (SSS), where skill information is transformed into vector representations and injected through a cache pathway instead of being appended as markdown text. Using a fixed base model (Qwen2.5-0.5B-Instruct), we compare three conditions across 100 skill-linked questions: C0 (no skills), C1 (markdown skill injection), and C2 (latent/cache skill injection with iterative projector training). Results show C1 remains best in absolute correctness, but C2 substantially improves over baseline while removing markdown context overhead. At the best C2 checkpoint (005), exact string matches occur in 25/100 queries versus 2/100 in C0. Performance peaks at iteration 005 and then regresses at 006, indicating a stability ceiling. These findings support SSS as a practical tradeoff for small-model, low-compute agent flows where context budget is a primary constraint.

1 Introduction

Skill files can be viewed as a practical extension of prompt engineering. As context windows increased, teams began attaching longer instruction artifacts ("skills") to user prompts so models could behave more consistently in task-specific domains. This approach is effective, especially in larger models with generous context budgets, and is now common in agentic workflows [1].

However, for smaller models, skill-file prompting can be expensive and fragile. Injecting skills as markdown consumes context window budget and increases semantic density in prompt tokens. In low-compute or latency-sensitive systems, this can become a bottleneck.

This paper investigates whether skill behavior can be transferred through latent representations instead of prompt text. The core idea is to preserve task-relevant skill information while reducing context overhead by injecting vectorized skill state into generation-time model internals. This framing is aligned with prior work on attention-based sequence modeling [8], continuous prompt methods [6, 5], and deep prompt optimization [7].

2 Hypothesis

2.1 Core Claim

Skill files are useful but fundamentally context-bound. Semantic Skill Space (SSS) hypothesizes that if skill semantics are encoded into model-compatible latent vectors and injected through a cache

pathway, then small models can retain useful skill behavior without paying markdown prompt overhead.

2.2 Formal Hypotheses

- H1: C2 (latent/cache injection) will outperform C0 (no skills) in correctness and final score.
- H2: C2 will maintain non-degeneracy score above 40/50 while removing markdown skill context from prompts.
- H3: C2 may increase deterministic exact-value behavior for some key-value style outputs.
- H4: C2 performance will have an optimization ceiling, with potential regression after excessive iterative updates.

Operationally, this paper treats "practically usable" as non-degeneracy $\geq 40/50$ on the 100-question evaluation set.

3 Methodology

3.1 Model and Dataset

- Base model: Qwen/Qwen2.5-0.5B-Instruct
- Evaluation set: 100 lightweight, skill-linked question/answer tests (one per skill file)
- Skill source: local markdown files in `skills/`

3.2 Experimental Conditions

- C0 (baseline): base model with no skill injection.
- C1 (markdown skills): same model with skill markdown inserted into prompt context.
- C2 (latent/cache): same model with no markdown skill text in prompt; skill information is embedded, projected, and injected through cache pathway.

All conditions use the same frozen base model to isolate the effect of skill delivery method.

3.3 Scoring

Each question is scored in two independent halves:

- Correctness: 0.0 to 0.5
- Non-degeneracy: 0.0 to 0.5

Run-level metrics:

- `correctness_out_of_50`
- `non_degeneracy_out_of_50`
- `final_score_out_of_100`

Degeneracy checks include repetitive loops, numeric-junk bursts, and malformed/garbled output patterns.

4 Implementation

4.1 Latent Skill Pathway (C2)

For each skill, the C2 pipeline applies:

1. Read skill markdown text.
2. Compute frozen hidden-state embedding for the skill text.
3. Mean-pool to a fixed representation.
4. Pass through a learned projector into KV-compatible tensors.
5. Inject projected state during generation.
6. Train projector only (base model frozen).
7. Save checkpoint, history, and results under `data/C2/<iteration>/`.

The injection design is conceptually related to prefix/prompt methods that influence attention states through learned continuous representations rather than raw discrete prompt text [6, 7].

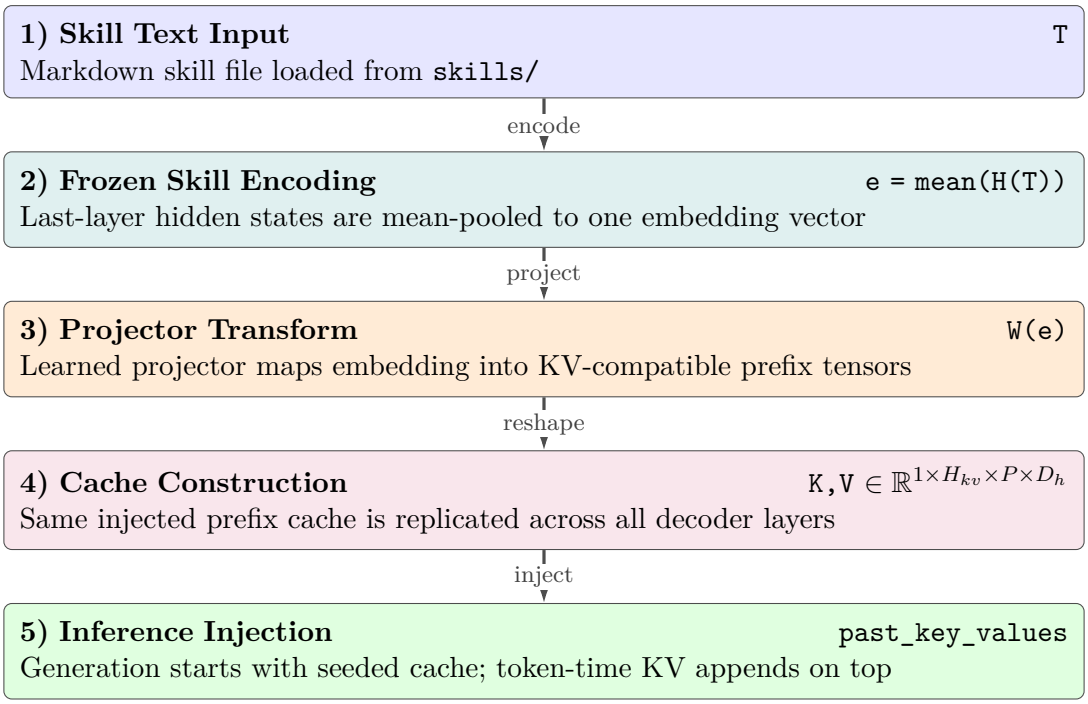

Figure 1: C2 cache pathway. Color-coded stages show the flow from raw skill markdown to generation-time KV cache seeding.

4.2 Cache Pathway Mechanism

The mechanism is prefix-tuning style at inference time: C2 does not prepend discrete virtual tokens to text, but directly seeds `past_key_values` with learned key/value tensors. The projector output has shape $[2, H_{kv}, P, D_h]$ where 2 denotes key/value, H_{kv} is key-value heads, P is prefix length, and

D_h is head dimension. This is converted to batched tensors $K, V \in \mathbb{R}^{1 \times H_{kv} \times P \times D_h}$ and replicated across all decoder layers as the initial cache state. During generation, the model appends token-time keys/values onto this seeded cache, while the attention mask is expanded by P so the injected prefix is attendable from the first generated token.

4.3 Iterative Training Setup

C2 uses iterative continuation where iteration $n+1$ starts from iteration n projector state. Stabilization settings used in this report:

- 3 epochs per iteration
- learning rate 1×10^{-4}
- prefix length = 2
- warmup steps = 100
- gradient clipping = 1.0
- alpha ramp from 0.05 to 0.25
- near-zero projector initialization

For implementation behavior around key-value cache handling and generation-time caching semantics, we follow the Transformer framework conventions documented in the Hugging Face Transformers cache documentation [4].

4.4 Implementation Details

Hyperparameters were chosen to stabilize projector-only training on CPU and small-model scale: short prefix length ($P = 2$) limits injected degrees of freedom; near-zero projector initialization keeps early KV perturbations gentle; alpha ramp ($0.05 \rightarrow 0.25$) gradually increases injection strength during training; warmup plus gradient clipping reduces early-step instability while preserving convergence speed.

5 Results

5.1 Overall Comparison

Table 1: Overall condition comparison

Condition	Correctness / 50	Non-degeneracy / 50	Final / 100
C0 (no skills)	4.0	46.0	50.0
C1 (markdown skills)	45.5	43.5	89.0
C2-001 (first)	1.5	19.5	21.0
C2-005 (best)	21.5	43.5	65.0
C2-006 (latest)	16.0	38.0	54.0

Table 2: C2 iterative progression

Iteration	Correctness / 50	Non-degeneracy / 50	Final / 100
001	1.5	19.5	21.0
002	10.0	29.0	39.0
003	18.5	40.0	58.5
004	21.0	40.0	61.0
005	21.5	43.5	65.0
006	16.0	38.0	54.0

5.2 C2 Iteration Progression

5.3 Trend Deltas

Table 3: Correctness trend deltas

Step	ΔCorrectness / 50
C0 → 001	-2.5
001 → 002	+8.5
002 → 003	+8.5
003 → 004	+2.5
004 → 005	+0.5
005 → 006	-5.5

Table 4: Non-degeneracy trend deltas

Step	ΔNon-degeneracy / 50
C0 → 001	-26.5
001 → 002	+9.5
002 → 003	+11.0
003 → 004	+0.0
004 → 005	+3.5
005 → 006	-5.5

5.4 Key Quantitative Findings

- C1 is strongest overall: 89.0 / 100.
- C2 best checkpoint (005) reaches 65.0 / 100, clearly above C0 (50.0).
- C2 improves rapidly from 001 → 005, then regresses at 006.
- C0 has high non-degeneracy (46.0/50) while C2-001 starts much lower (19.5/50), indicating that naive early injection initially harms output quality before training recovers stability.
- Relative to C0, C2-005 adds +17.5 correctness points and reduces non-degeneracy gap to parity with C1 (43.5).

Full per-question outputs are available in repository artifacts under `data/` (including C0, C1, and C2 iteration result files) [3].

5.5 Deterministic Exact-Value Behavior

At checkpoint C2-005, exact answer-string matches appear in 25/100 queries, versus 2/100 in C0 and 44/100 in C1. The C2 exact-match gains over C0 are concentrated in numeric values (14/25) and boolean flags (8/25), with smaller gains for short literal strings (3/25).

Table 5: Example deterministic outputs from C2-005 (exact expected-value matches)

Skill key	Type	Expected	C2-005 output
`max_requests_per_minute`	numeric value	60	60
`cache_ttl_seconds`	numeric value	300	300
`enable_response_cache`	boolean flag	true	true
`compression_codec`	short literal	gzip	gzip
`retry_budget`	short literal	3 attempts	3 attempts

6 Discussion

The central tradeoff is clear: markdown skill prompting (C1) maximizes correctness, while latent/cache injection (C2) trades some peak accuracy for context efficiency.

6.1 Why This Still Matters

Even though C2 remains below C1 in absolute score, C2 does not require skill markdown to occupy the inference prompt. For constrained systems, this matters operationally: less context pressure, fewer prompt tokens, and lower risk of prompt-window contention.

C2 also showed deterministic exact-value outputs in some skill-specific key-value tasks, especially numeric and boolean configuration fields. This is notable because deterministic value retrieval is often desirable in agent pipelines, even when overall natural-language quality is weaker than full prompt conditioning.

6.2 Why Regression Appears

The 005 $\rightarrow$ 006 drop suggests an optimization ceiling. Plausible causes include overfitting in projector space, latent drift relative to frozen base geometry, and cumulative instability from repeated continuation updates.

This pattern is consistent with the broader literature that learned soft-conditioning methods can be highly effective but sensitive to optimization setup and scale [5, 7]. Related system work on prompt-state reuse also highlights that cache-side methods can reduce repeated context cost but introduce new control and stability considerations [2].

Practically, this supports a checkpoint-selection strategy: choose best validated iteration (here, 005) rather than latest.

7 Limitations

- Single primary base model and scale regime (0.5B).
- Limited hardware envelope and training budget.
- No large-model replication in this study.
- Task set focuses on lightweight skill QA; broader domains may behave differently.

These results should be interpreted as strong pilot evidence rather than final generalization across all architectures.

8 Conclusion

This study supports the SSS hypothesis in its practical form: latent/cache skill injection is a viable method for small models when context budget is constrained. It does not surpass markdown skill prompting on peak correctness, but it substantially improves over baseline without context overhead and can deliver deterministic exact-value behavior in some cases.

The observed rise through C2-005 and decline at C2-006 indicates that iterative projector training has a useful zone and a degradation zone. For deployment, this implies tuning with early-stop checkpoint selection rather than unlimited continuation.

Overall, SSS is promising for low-compute agent flows where full skill-file prompting is not feasible. Code and raw outputs are available in the project repository [3].

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Gilwoo Gim, Jaewoo Kim, Taehyeon Lee, et al. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2024.

[3] Ethan Blair. Semantic Skill Space (SSS) repository and release artifacts. GitHub: `https://github.com/i3T4AN/Semantic-skill-space`; Zenodo DOI: `https://doi.org/10.5281/zenodo.18830835`, 2026.

[4] Hugging Face. Hugging Face Transformers documentation: Caching. `https://huggingface.co/docs/transformers/main/cache_explanation`, 2026. Accessed: 2026-03-02.

[5] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *EMNLP*, 2021.

[6] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL-IJCNLP*, 2021.

[7] Xiao Liu, Kaixuan Ji, Yicheng Fu, Wing-Kwong Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. In *ACL*, 2022.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.